

HUNT EYE

Autonomous UAV Target Detection, Tracking & Interception

Phase-by-Phase Implementation Plan

6

Phases

15

Build Steps

16

Weeks

100%

Open Source

Python Powered • RTX 3050 • Windows 11 + WSL2 • v1.0 — 2025

Table of Contents

PHASE 0	Environment Setup	Steps 0A–0E · Week 1–2
PHASE 1	Project Skeleton	Steps 1–3 · Week 3–4
PHASE 2	Vision — Making the Drone See	Steps 4–6 · Week 5–7
PHASE 3	Tracking — Locking On	Steps 7–8 · Week 8–9
PHASE 4	Control — Making It Move	Step 9 · Week 10–11
PHASE 5	AI Brain	Steps 10–12 · Week 12–14
PHASE 6	Re-ID, Safety & Real Drone	Steps 13–15 · Week 15–16

Project Overview

Hardware	RTX 3050 Laptop GPU + Windows 11 + WSL2
Language	Python 3.10 — all tools have Python bindings
Total Steps	15 Build Steps across 6 Phases
Timeline	~16 weeks (realistic: extend each phase by 1.5x for beginners)
Goal	One command launches a drone that autonomously hunts a target
Deployment	AirSim simulation first → real drone via one config line change

PHASE 0

Environment Setup

Getting Your Computer Ready · Weeks 1–2

Before writing a single line of code, your computer needs the right foundation. Think of this as building your workshop before construction begins. Do it once — never again.

0A	Install WSL2 — Windows Subsystem for Linux	~1 hr
0B	Install Python 3.10 + CUDA 11.8	~1 hr
0C	Install AirSim (Pre-Built Binary)	~30 min
0D	Install PX4 SITL inside WSL2	~2 hrs
0E	Create Python Virtual Environment	~30 min

Step 0A — Install WSL2

WSL2 lets Windows run a full Linux OS in a window. PX4 SITL only works on Linux — WSL2 is the bridge. No dual-boot needed.

- Open PowerShell as Administrator (right-click Start menu)

```
wsl --install
```

- Restart when prompted. Ubuntu 22.04 opens — create a username and password.

```
wsl --list --verbose # Should show: Ubuntu Running 2
```

- If you see version 1 instead of 2 — run: `wsl --set-version Ubuntu 2`

Step 0B — Python 3.10 + CUDA 11.8

CUDA is Nvidia's software that lets Python use your GPU. Without it YOLOv8 runs at ~5 fps on CPU. With CUDA on your RTX 3050 it runs at 30+ fps.

- Download Python 3.10.x 64-bit from python.org — CHECK 'Add to PATH'
- Download CUDA 11.8 from developer.nvidia.com/cuda-downloads → Express Install

```
python --version          # Python 3.10.x
nvcc --version             # Cuda compilation tools, release 11.8
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118
python -c "import torch; print(torch.cuda.is_available())" # Must print: True
```

Step 0C — AirSim Binary

AirSim is your 3D world — a physics-accurate simulation environment. We use the pre-built Blocks binary to avoid installing 50 GB of Unreal Engine.

- Go to: github.com/microsoft/AirSim/releases → Download AirSimNH.zip

- Extract to C:\AirSim\ → double-click AirSimNH.exe → press 1 for drone mode

```
pip install airsimsim
python -c "
import airsimsim
client = airsimsim.MultirotorClient()
client.confirmConnection()
client.enableApiControl(True)
client.armDisarm(True)
client.takeoffAsync().join()
print('Drone took off!')
```

✓ **MILESTONE: AirSim open, Python connects, drone takes off and hovers.**

Step 0D — PX4 SITL inside WSL2

PX4 is the operating system inside every serious drone. SITL runs the real firmware on your computer instead of a chip — handling stability, motor speeds, and flight modes.

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y git cmake python3-pip ninja-build
git clone https://github.com/PX4/PX4-Autopilot.git --recursive
cd PX4-Autopilot && bash ./Tools/setup/ubuntu.sh
make px4_sitl gazebo-classic_iris
# Success = INFO [px4] Startup script returned successfully
```

■ If build fails: `sudo apt install -y gcc-arm-none-eabi`

Step 0E — Virtual Environment

A virtual environment is an isolated folder for one project's packages — prevents conflicts between projects.

```
cd C:\ && mkdir hunt_eye && cd hunt_eye
python -m venv venv
venv\Scripts\activate
pip install airsimsim ultralytics opencv-python timm torch torchvision ^
stable-baselines3 gymnasium pymavlink PyQt6 numpy geopy ^
osmnx torchreid rasterio tensorboard
```

✓ **MILESTONE: Virtual environment created. All 16 packages installed. Workshop is ready.**

PHASE 1

Project Skeleton

The Foundation · Weeks 3–4

Build the empty frame of the house before adding walls and furniture. Every file is a placeholder we fill in coming phases. The architecture is correct from Day 1.

- | | | |
|---|---|--------|
| 1 | Create Folder Structure + config.yaml | ~2 hrs |
| 2 | SharedState + HAL + SystemOrchestrator | ~3 hrs |
| 3 | CameraStream + Dashboard (First Visual) | ~3 hrs |

Step 1 — Folder Structure + config.yaml

```

hunt_eye/
  main.py           # THE one command that starts everything
  config.yaml       # all settings – change mode:sim to mode:real here
  requirements.txt
  core/
    orchestrator.py  shared_state.py  hal.py
    event_bus.py     safety.py
  simulation/
    airsim_client.py terrain_loader.py
  perception/
    camera_stream.py detector.py depth_estimator.py
  tracking/
    tracker.py target_filter.py reid_engine.py
  ai/
    gym_env.py agent.py planner.py
  control/
    mavlink_interface.py
  ui/
    dashboard.py

```

Key config.yaml excerpt:

```

mode: sim           # CHANGE TO 'real' FOR REAL DRONE
yolo:
  model: "yolov8n.pt"
  confidence: 0.45
  target_classes: ["person"]
safety:
  max_velocity_ms: 5.0
  max_altitude_m: 30.0
  geofence_radius_m: 100.0
terrain:
  bbox: [72.8, 18.9, 72.95, 19.05] # Mumbai bounding box

```

Step 2 — SharedState + HAL

SharedState is the central whiteboard every thread reads and writes. HAL (Hardware Abstraction Layer) hides the difference between simulation and real drone — everything above it is identical either way.

```

# core/shared_state.py
@dataclass

```

```
class SharedState:
    lock: threading.RLock
    uav_position: np.ndarray # [x, y, z] metres
    latest_frame: np.ndarray # current camera image
    detections: list # YOLOv8 output boxes
    active_target_id: int = -1 # -1 = no target
    target_bbox: tuple = None
    cost_map: np.ndarray = None
    planned_path: list = []
    system_mode: str = "idle" # idle/tracking/returning/emergency
```

Step 3 — CameraStream + Dashboard

After this step you see a live camera feed from inside AirSim in a PyQt6 desktop window. This proves the entire pipeline from AirSim → Python → screen is working.

```
# perception/camera_stream.py
class CameraStream:
    def stream_loop(self):
        while True:
            frame = self.hal.get_frame()
            with self.state.lock:
                self.state.latest_frame = frame
            time.sleep(1/30) # 30 Hz
```

✓ **MILESTONE:** python main.py → drone hovers in AirSim, PyQt6 window shows live camera feed.

PHASE 2

Vision — Making the Drone See

Steps 4–6 · Weeks 5–7

Now we give the drone eyes — detect objects in real-time, estimate distances from a single camera, and pre-load a terrain cost map so the planner knows what to avoid.

4

YOLOv8 Object Detector

~2 hrs

5

MiDaS Depth Estimation

~2 hrs

6

Terrain Loader — Cost Map from Real Data

~4 hrs

Step 4 — YOLOv8 Object Detector

YOLOv8n (nano) is the fastest YOLO model — ~35 fps at 640x480 with GPU. We filter by class before tracking so the tracker never sees irrelevant objects.

```
from ultralytics import YOLO
class Detector:
    def __init__(self, config, state):
        self.model = YOLO(config["model"]) # downloads yolov8n.pt automatically
    def detect(self, frame):
        results = self.model(frame, conf=self.conf, verbose=False)
        detections = []
        for r in results:
            for box in r.bboxes:
                cls_name = self.model.names[int(box.cls)]
                if cls_name in self.target_classes:
                    detections.append({"bbox": box.xyxy[0].tolist(),
                                      "confidence": float(box.conf)})
        return detections
```

Step 5 — MiDaS Depth Estimation

MiDaS estimates how far away each pixel is from a single RGB camera — no expensive stereo camera needed. Output is a depth map where bright = far, dark = near.

```
self.model = torch.hub.load("intel-isl/MiDaS", "MiDaS_small")
# Runs on CUDA automatically — ~8ms per frame on RTX 3050
depth_np = depth.cpu().numpy() # 2D array: same shape as input frame
```

Step 6 — Terrain Loader (Cost Map)

GDAL reads real terrain elevation data. osmnx downloads building footprints from OpenStreetMap. Combined into a 2D cost map: 0 = open air, 100 = building/mountain. Pre-loaded once at startup — A* reads it every planning cycle.

```
# Mumbai bounding box: [72.8, 18.9, 72.95, 19.05]
buildings = ox.features_from_bbox(..., tags={"building": True})
slope_cost = np.sqrt(gradient[0]**2 + gradient[1]**2) * 10
cost_map = np.clip(slope_cost + obstacle_map * 100, 0, 100)
```

✓ **MILESTONE:** Dashboard shows bounding boxes on detected people + depth map + coloured cost map.

PHASE 3

Tracking — Locking On

Steps 7–8 · Weeks 8–9

Detection finds ALL targets every frame. Tracking locks onto ONE and never lets go. This is the 'selective' in Selective Target Tracking.

7

ByteTrack Multi-Object Tracker

~3 hrs

8

TargetFilter — Selective Lock-On Logic

~2 hrs

Step 7 — ByteTrack

ByteTrack remembers which bounding box in frame 50 is the same person as frame 49. Uses Kalman filter + IoU matching. Assigns a permanent integer ID to each person. Handles occlusion better than older trackers by using both high and low confidence detections.

```
# Install:
git clone https://github.com/ifzhang/ByteTrack.git
cd ByteTrack && pip install -r requirements.txt && pip install -e .

# Usage:
self.tracker = BYTETracker(
    track_thresh=0.5,    # min confidence to track
    track_buffer=30,     # frames to keep a lost track alive
    match_thresh=0.8     # IoU threshold for matching
)
```

Step 8 — TargetFilter

ByteTrack gives all tracks. TargetFilter picks ONE and commits to it. On first detection picks highest-confidence track. Every subsequent frame verifies the target still matches. If lost, triggers re-ID mode.

```
if active_id == -1:    # No target yet
    best = max(tracks, key=lambda t: t.score)
    state.active_target_id = best.track_id
    print(f"Locked onto track ID {best.track_id}")
else:                 # Already tracking
    matched = [t for t in tracks if t.track_id == active_id]
    if not matched:
        state.system_mode = "reacquiring"    # triggers OSNet re-ID
```

✓ **MILESTONE: One target tracked with persistent ID — doesn't change when others enter frame.**

PHASE 4

Control — Making It Move

Step 9 · Weeks 10–11

The drone stops being a spectator and becomes a hunter. We implement a PID controller — a classic algorithm from the 1940s still used in everything from thermostats to rockets.

9

MAVLink Interface + PID Controller

~4 hrs

Step 9 — MAVLink Interface + PID Controller

How PID works for drone pursuit:

P — Proportional	Target left of centre → fly left. Further = faster. Main correction force.
I — Integral	Keeps slightly missing? Gradually increase correction to remove persistent drift.
D — Derivative	Correcting too fast? Slow down to prevent overshoot. Damping force.

```
# Tune these three values to adjust drone behaviour:
self.Kp = 0.002    # proportional – main correction strength
self.Ki = 0.0001   # integral – drift correction
self.Kd = 0.001    # derivative – overshoot damping

error_x = target_cx - frame_cx      # + = target right of centre
vx = Kp*error + Ki*integral + Kd*delta_error
vx = max(-2.0, min(2.0, vx))        # cap speed for safety
self._send_velocity(vx, vy, -0.1, 0) # -0.1 vz = slowly descend
```

✓ MILESTONE: Drone moves in AirSim to keep target centred. Target walks left — drone follows left.

PHASE 5

AI Brain

Steps 10–12 · Weeks 12–14

PID works but is dumb — it only reacts, never plans. If the target runs around a building, PID flies into the wall. Reinforcement Learning trains an AI to anticipate, plan, and optimise.

10	Gymnasium Environment Wrapper	~4 hrs
11	PPO Training (leave overnight)	~8 hrs active + training time
12	A* Path Planner on Cost Map	~3 hrs

Step 10 — Gymnasium Environment Wrapper

Before training an RL agent we wrap our simulation as a Gymnasium environment — defining what the AI sees (observations), what it can do (actions), and how it scores (rewards).

```
# Observations: 8 numbers the AI sees each step
observation_space = gym.spaces.Box(low=-1, high=1, shape=(8,))

# Actions: [vx, vy, vz, yaw_rate] all in [-1, 1], scaled to 5 m/s
action_space = gym.spaces.Box(low=-1, high=1, shape=(4,))

# Reward function:
# + 1.0 for target centred in frame
# + 0.5 for being close (large bounding box)
# - 2.0 heavy penalty for losing target
```

Step 11 — PPO Training

PPO (Proximal Policy Optimization) trains by running the simulation for millions of steps. Each step the AI tries an action, gets a reward, and updates its policy. Training 1M steps takes 2–4 hours on RTX 3050 — leave AirSim running overnight.

```
model = PPO("MlpPolicy", env, verbose=1,
            learning_rate=3e-4, n_steps=2048,
            batch_size=64, gamma=0.99,
            tensorboard_log="./logs/")
model.learn(total_timesteps=1_000_000)
model.save("models/ppo_hunt_eye")

# Monitor training live:
tensorboard --logdir ./logs/
```

■ RL convergence is NOT guaranteed. Reward shaping and hypertuning may take many attempts.

Step 12 — A* Path Planner

A* (A-star) finds the shortest safe path from A to B on the cost map, avoiding high-cost cells (buildings, terrain). RL agent says 'go to this coordinate' — A* figures out the safest route.

```
# A* returns list of (row, col) waypoints
# Cells with cost >= 90 are treated as blocked (solid obstacles)
# Diagonal moves allowed – 8-directional grid
```

```
step_cost = sqrt(dx**2+dy**2) * (1 + cost_map[nb]/10)
# Heuristic = straight-line distance to goal
```

✓ **MILESTONE: RL policy replaces PID. Drone navigates around obstacles. A* path shown on dashboard.**

PHASE 6

Re-ID, Safety & Real Drone

Steps 13–15 · Weeks 15–16

Final phase: re-acquisition when target hides, mandatory safety checks before every command, and the sim-to-real switch. After this, Hunt Eye is complete.

13	OSNet Re-Identification Engine	~3 hrs
14	Safety Layer (Non-Negotiable)	~3 hrs
15	Sim-to-Real Switch	~2 hrs + flight testing

Step 13 — OSNet Re-Identification

When the target fully disappears and reappears, ByteTrack gives them a new ID. OSNet creates a 512-number visual fingerprint of the target's appearance — clothes, colours, body shape. On re-acquisition, compares fingerprints using cosine similarity. If similarity > 0.65 threshold: that is our target — re-lock and continue.

```
self.model = torchreid.models.build_model(
    name="osnet_x1_0", num_classes=1000, pretrained=True)

# Cosine similarity match:
sim = dot(embed_A, embed_B) / (norm(embed_A) * norm(embed_B))
if sim >= self.threshold: # default 0.65
    print(f"Target re-acquired! Similarity: {sim:.3f}")
```

Step 14 — Safety Layer

The safety layer is the LAST thing checked before EVERY command reaches the drone. No command bypasses it. Five hard rules:

Heartbeat check	If connection lost > 2 seconds → land immediately
Velocity cap	Drone cannot exceed 5 m/s regardless of AI command
Altitude cap	Drone cannot go above 30 metres
Geofence	Cannot leave 100m radius from launch point
Emergency land	Any module sets system_mode = 'emergency' → instant land

Step 15 — Sim-to-Real Switch

Everything worked in simulation. Now flip the switch. Because HAL hides all differences, the switch is literally one word:

```
# config.yaml
mode: real # was: sim
real:
  serial_port: "/dev/ttyUSB0" # or COM3 on Windows
  baud_rate: 57600

# Then just run:
python main.py --config config.yaml
```

Component	Transfers?	Notes
YOLOv8	YES	May need fine-tuning on real lighting
ByteTrack	YES	Pure math — works on any video stream
OSNet Re-ID	YES	Camera-agnostic
A* Planner	YES	Same cost map
Safety Layer	YES	More important in real mode
RL Policy	MOSTLY	Domain randomisation handles physics gap
Camera	ONE-TIME	Each real camera needs lens calibration
MAVLink	CHANGED	Serial port instead of UDP — RealBackend handles it

✓ **MILESTONE: FINAL: drone takes off, locks on, follows, re-acquires, all with safety checks.**

Appendix — Quick Reference

All Install Commands

```
pip install airsims ultralytics opencv-python timm torch torchvision torchaudio ^
stable-baselines3 gymnasium pymavlink PyQt6 numpy geopy ^
osmnx torchreid rasterio tensorboard

# ByteTrack (not on pip):
git clone https://github.com/ifzhang/ByteTrack.git
cd ByteTrack && pip install -r requirements.txt && pip install -e .

# PX4 SITL (inside WSL2):
git clone https://github.com/PX4/PX4-Autopilot.git --recursive
cd PX4-Autopilot && bash ./Tools/setup/ubuntu.sh
make px4_sitl gazebo-classic-iris
```

Run Commands

```
# 1. Start AirSim first:
C:\AirSim\AirSimNH.exe

# 2. Activate env + run Hunt Eye:
venv\Scripts\activate
python main.py --config config.yaml

# 3. Train RL agent (separate session):
python train_agent.py

# 4. Monitor training:
tensorboard --logdir ./logs/

# 5. Switch to real drone:
# Edit config.yaml: mode: real
python main.py --config config.yaml
```

Thread Frequencies

Thread	Speed	Why This Speed
camera_stream	30 Hz	Camera is 30 fps — grab every frame
perception_loop	15 Hz	YOLOv8 takes ~33ms on RTX 3050
tracking_loop	15 Hz	Runs right after perception
ai_loop	10 Hz	RL policy query — 10x/sec is enough
planner_loop	5 Hz	A* only needed when waypoint changes
control_loop	20 Hz	Flight controller needs frequent commands
dashboard	10 Hz	Human eye can't process faster anyway

Glossary

AirSim	3D simulation tool by Microsoft — realistic video game your code controls
CUDA	Nvidia software enabling Python to use GPU for fast math
DEM	Digital Elevation Model — terrain height at each GPS point
Geofence	Invisible GPS boundary the drone cannot cross
Gymnasium	Standard RL training environment interface by OpenAI

HAL	Hardware Abstraction Layer — hides sim vs real drone differences
IoU	Intersection over Union — measures bounding box overlap (0-1)
MAVLink	Micro Air Vehicle Link — communication protocol all drones speak
OSNet	Neural network identifying people by appearance (clothes, body shape)
PID	Proportional-Integral-Derivative — classic stabilisation formula
PPO	Proximal Policy Optimization — reliable RL training algorithm
Re-ID	Re-Identification — recognising a person after leaving camera view
RL	Reinforcement Learning — training AI through trial, reward, repetition
SITL	Software In The Loop — flight firmware running on a computer
YOLO	You Only Look Once — object detection AI at 30+ fps

Hunt Eye — 100% open source. 100% free. Your job is to connect the pieces.